

SDITO Lab PC-CS6G4

Name : Debatwik Santra

Roll No : CSE/23/110

Exam Roll No : 11600223047

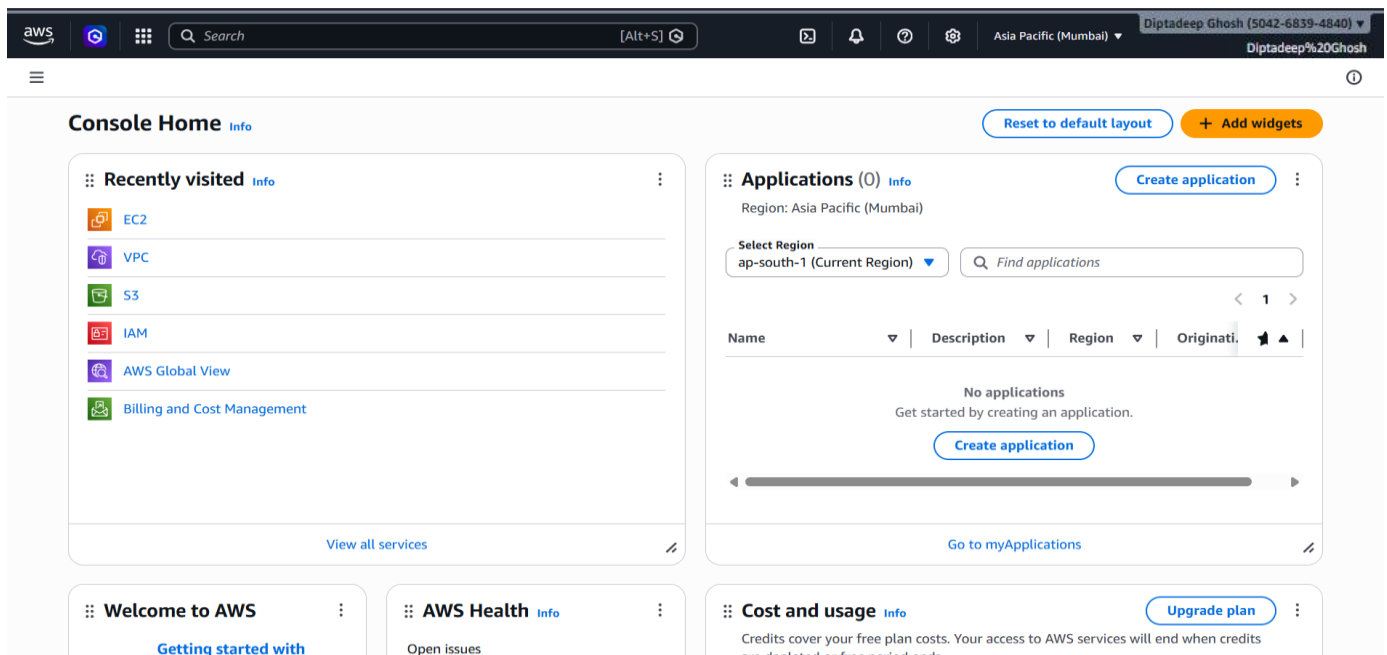
Assignment 12

Problem Statement : Deploy and Run the project in AWS without using the Port.

Solution:

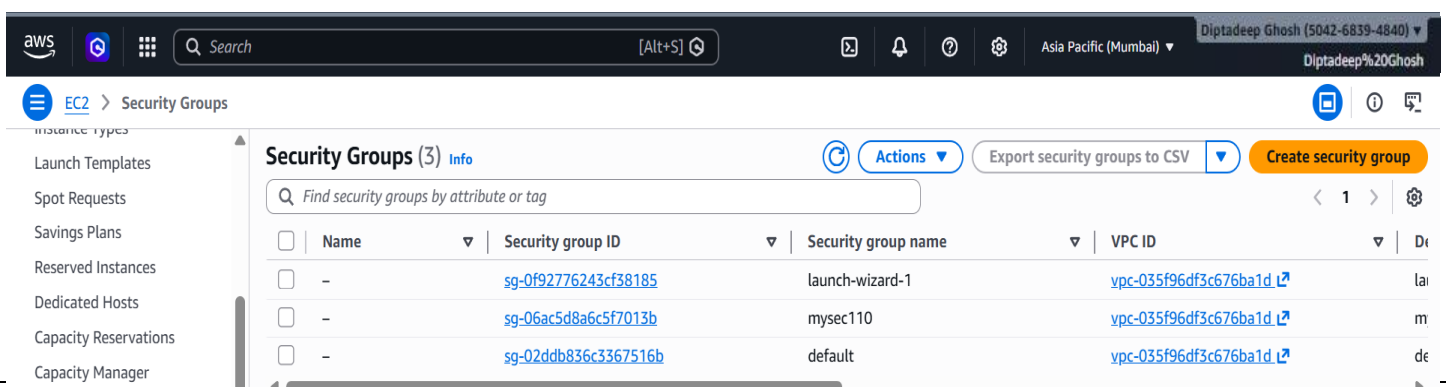
STEP 1: Create a New Security Group(Skip this step if security group has already been created)

1. Login to AWS Console



2. Go to EC2 Dashboard

3. Click Security Groups



4. Click Create Security Group

5. Enter:

- Security group name: **DebSec110**
- Description: **DebSec110**
- Select default VPC

aws [Search] [Alt+S] Asia Pacific (Mumbai) Diptadeep Ghosh (5042-6839-4840) Diptadeep%20Ghosh

EC2 > Security Groups > Create security group

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

Inbound rules [Info](#)

This security group has no inbound rules.

[Add rule](#)

Outbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Destination Info	Description - optional Info
---------------------------	-------------------------------	---------------------------------	----------------------------------	---

Add Inbound Rules

aws [Search] [Alt+S] Asia Pacific (Mumbai) Diptadeep Ghosh (5042-6839-4840) Diptadeep%20Ghosh

EC2 > Security Groups > Create security group

Inbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
SSH	TCP	22	Cust...		Delete

Search for a rule type to add.

- Custom TCP
- Custom UDP
- Custom ICMP - IPv4
- Custom Protocol
- All TCP
- All UDP
- All ICMP - IPv4
- All ICMP - IPv6
- All traffic
- SSH**
- SMTP
- DNS (UDP)
- DNS (TCP)

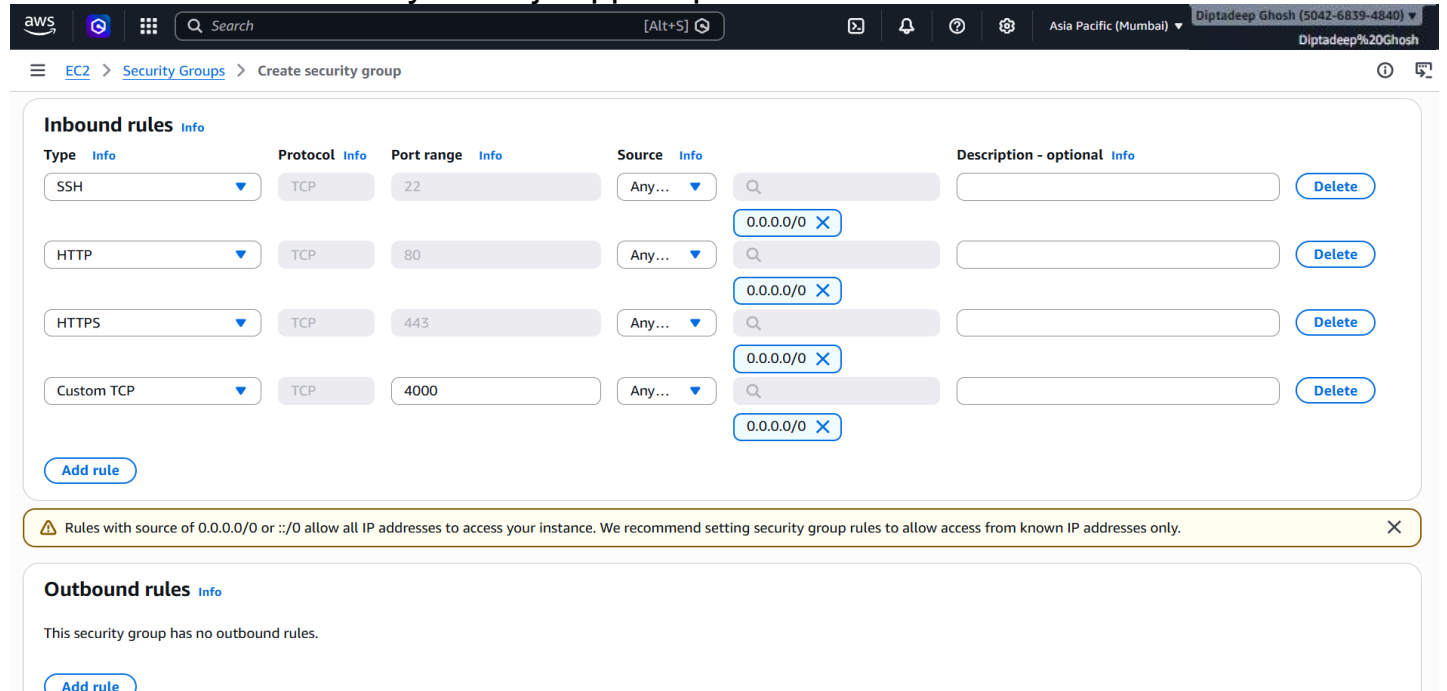
[Cancel](#) [Create security group](#)

Add the following rules:

Type	Port	Source
SSH	22	0.0.0.0/0
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0
Custom TCP	4000	0.0.0.0/0

Why 4000?

Because I want to run my Node.js app on port 4000.



Inbound rules Info

Type	Protocol	Port range	Source	Description - optional	Actions
SSH	TCP	22	Any...		Delete
HTTP	TCP	80	Any...		Delete
HTTPS	TCP	443	Any...		Delete
Custom TCP	TCP	4000	Any...		Delete

[Add rule](#)

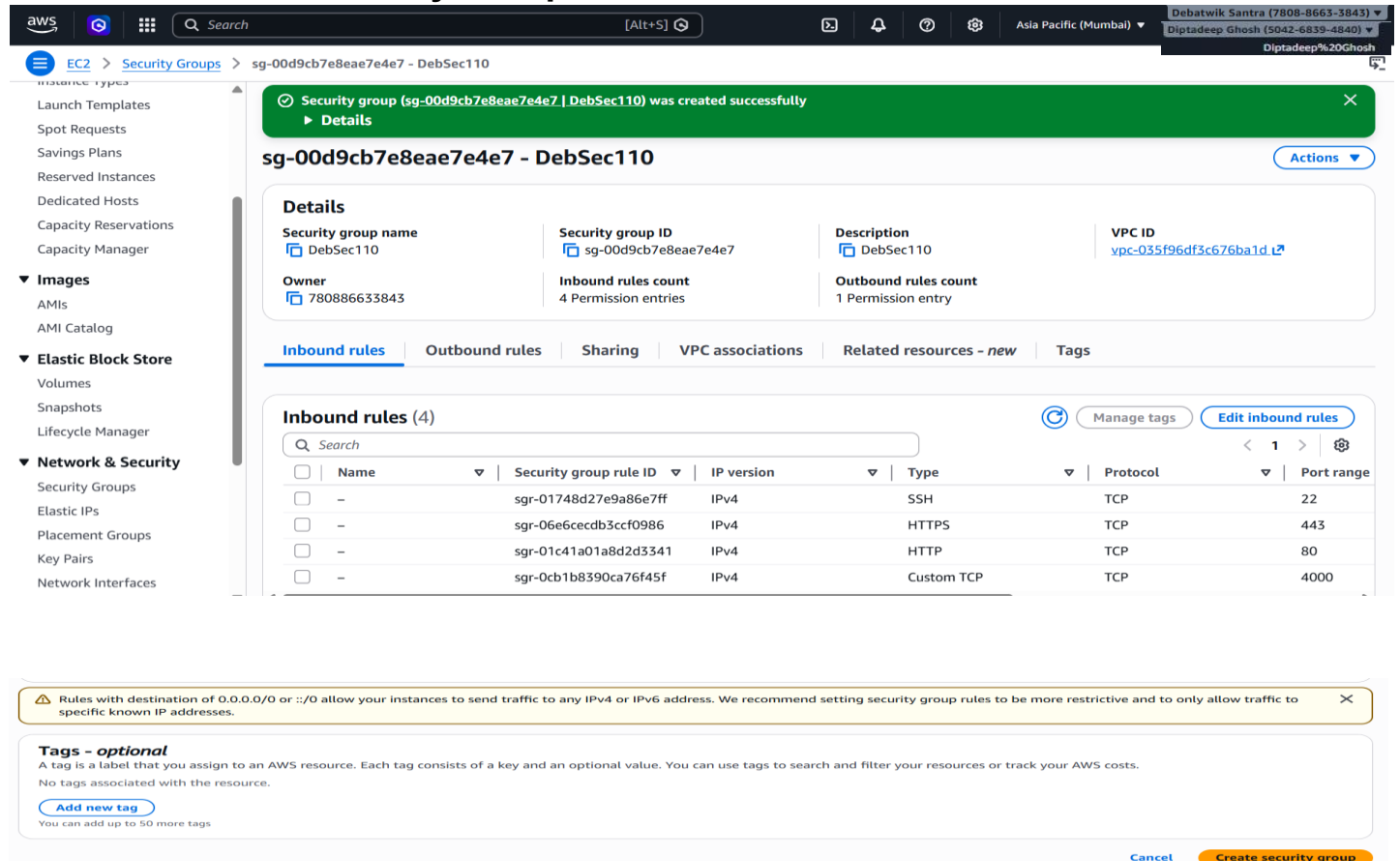
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Outbound rules Info

This security group has no outbound rules.

[Add rule](#)

➤ Click Create Security Group



Security group (sg-00d9cb7e8eae7e4e7 | DebSec110) was created successfully

sg-00d9cb7e8eae7e4e7 - DebSec110

Details

Security group name DebSec110	Security group ID sg-00d9cb7e8eae7e4e7	Description DebSec110	VPC ID vpc-035f96df3c676ba1d
Owner 780886633843	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (4)

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-01748d27e9a86e7ff	IPv4	SSH	TCP	22
-	sgr-06e6cecd3ccf0986	IPv4	HTTPS	TCP	443
-	sgr-01c41a01a8d2d3341	IPv4	HTTP	TCP	80
-	sgr-0cb1b8390ca76f45f	IPv4	Custom TCP	TCP	4000

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs. No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

[Cancel](#) [Create security group](#)

➤ Security group created successfully.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Diptadeep Ghosh (5042-6839-4840) Diptadeep%20Ghosh

EC2 > Security Groups

Instance types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

Security group (sg-00d9cb7e8ae7e4e7 | DebSec110) was created successfully
Details

Security Groups (1/4) Info Actions Export security groups to CSV Create security group

Find security groups by attribute or tag

	Name	Security group ID	Security group name	VPC ID
☐	-	sg-0f92776243cf38185	launch-wizard-1	vpc-035f96df3c676ba1d
☒	-	sg-00d9cb7e8ae7e4e7	DebSec110	vpc-035f96df3c676ba1d
☐	-	sg-06ac5d8a6c5f7013b	mysec110	vpc-035f96df3c676ba1d
☐	-	sg-02ddb836c3367516b	default	vpc-035f96df3c676ba1d

sg-00d9cb7e8ae7e4e7 - DebSec110

Details Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Details

Security group name DebSec110	Security group ID sg-00d9cb7e8ae7e4e7	Description DebSec110	VPC ID vpc-035f96df3c676ba1d
Owner 780886633843	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

STEP 2: Launch EC2 Instance

1. Go to **EC2** → **Instances**
2. Click **Launch Instance**

EC2 > Instances

EC2 Dashboard AWS Global View Events

▼ Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots

Instances Info Connect Instance state Actions Launch instances

Saved filter sets Choose filter set Find Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
No instances You do not have any instances in this region Launch instances							

Select an instance

Configuration:

- Name: **assg12**
- AMI: Ubuntu 24.04
- Instance Type: t3.micro
- Key pair: Select your existing key pair
- Network Settings:
 - Enable Auto-assign Public IP
 - Select existing security group → **keyAWS**

EC2 > Instances > Launch an instance

Name and tags

Name

assg12

Add additional tags

Application and OS Images (Amazon Machine Image)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.

Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type

ami-05d2d839d4f73aafb (64-bit (x86)) / ami-0f6c99577b93a642b (64-bit (Arm))

Virtualization: hvm - ENA enabled: true - Root device type: ebs

Free tier eligible

Description

Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

Architecture

AMI ID

Publish Date

Username

amd64

ami-05d2d839d4f73aafb

2026-03-14

ubuntu

Verified provider

Summary

Number of instances

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...read more

ami-05d2d839d4f73aafb

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

EC2 > Instances > Launch an instance

Key pair (login)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

keyAWS

Create new key pair

Network settings

Network

vpc-035f96df3c676ba1d

Subnet

No preference (Default subnet in any availability zone)

Auto-assign public IP

Enable

Firewall (security groups)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Common security groups

Select security groups

DebSec110 sg-00d9cb7e8ae7e4e7

VPC: vpc-035f96df3c676ba1d

Compare security group rules

Summary

Number of instances

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...read more

ami-05d2d839d4f73aafb

Virtual server type (instance type)

t3.micro

Firewall (security group)

DebSec110

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

STEP 3: Add User Data Script

In Advanced Details → User Data, add:

```
#!/bin/bash
apt-get update -y
apt-get upgrade -y
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
```

EC2 > Instances > Launch an instance

Allow tags in metadata

Select

User data - optional

Upload a file with your user data or enter it in the field.

Choose file

```
#!/bin/bash
apt-get update -y
apt-get upgrade -y
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
```

☐ User data has already been base64 encoded

Summary

Number of instances1

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...read more
ami-05d2d839d4f73aafb

Virtual server type (instance type)
t3.micro

Firewall (security group)
DebSec110

Storage (volumes)
1 volume(s) - 8 GiB

CancelLaunch instancePreview code

What this does:

- Updates system
 - Installs Nginx
 - Installs Git
 - Installs Node.js
- Click **Launch Instance**

EC2 > Instances > Launch an instance

Success
Successfully initiated launch of instance (i-0e7e1830ec5eb29ee)

STEP 4: Connect to EC2

1. Go to Instances
2. Select your instance
3. Click **Connect**
4. Use **EC2 Instance Connect**
5. Username: ubuntu
6. Click **Connect**

EC2 > Instances > i-0e7e1830ec5eb29ee > Connect to instance

Connect

Connect to an instance using the browser-based client.

Instance ID
i-0e7e1830ec5eb29ee (assg12)

VPC ID
vpc-035f96df3c676ba1d

Security groups
sg-00d9cb7e8eae7e4e7 (DebSec110)

IAM role
-

EC2 Instance Connect

SSM Session Manager

SSH client

EC2 serial console

Instance ID
i-0e7e1830ec5eb29ee (assg12)

Connection type

Connect using a Public IP
Connect using a public IPv4 or IPv6 address

Connect using a Private IP
Connect using a private IP address and a VPC endpoint

Public IPv4 address

13.127.19.2

IPv6 address
-

Username

Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ubuntu.

ubuntu

Note: In most cases, the default username, ubuntu, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

CancelConnect

➤ Terminal opens successfully.

STEP 5: Clone GitHub Repository

Inside EC2 terminal:

```
git clone https://github.com/debatwiksantra/awslab.git
cd awslab
ls
```

```
ubuntu@ip-172-31-41-95:~$ git clone https://github.com/debatwiksantra/awslab.git
Cloning into 'awslab'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 15 (delta 5), reused 4 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 4.15 KiB | 1.04 MiB/s, done.
Resolving deltas: 100% (5/5), done.
ubuntu@ip-172-31-41-95:~$ cd awslab
ubuntu@ip-172-31-41-95:~/awslab$ ls
index.js  package.json
ubuntu@ip-172-31-41-95:~/awslab$
```

STEP 6: Install Dependencies

```
npm install
```

STEP 7: Run the Application

If your entry file is index.js:

```
node index.js
```

```
ubuntu@ip-172-31-41-95:~/awslab$ npm install

added 224 packages, and audited 225 packages in 22s

26 packages are looking for funding
  run `npm fund` for details

1 low severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.12.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
npm notice To update run: npm install -g npm@11.12.1
npm notice
ubuntu@ip-172-31-41-95:~/awslab$ node index.js
Started server
```

STEP 8: Access Application in Browser

1. Copy Public IPv4 address

13.127.19.2

2. Open browser:

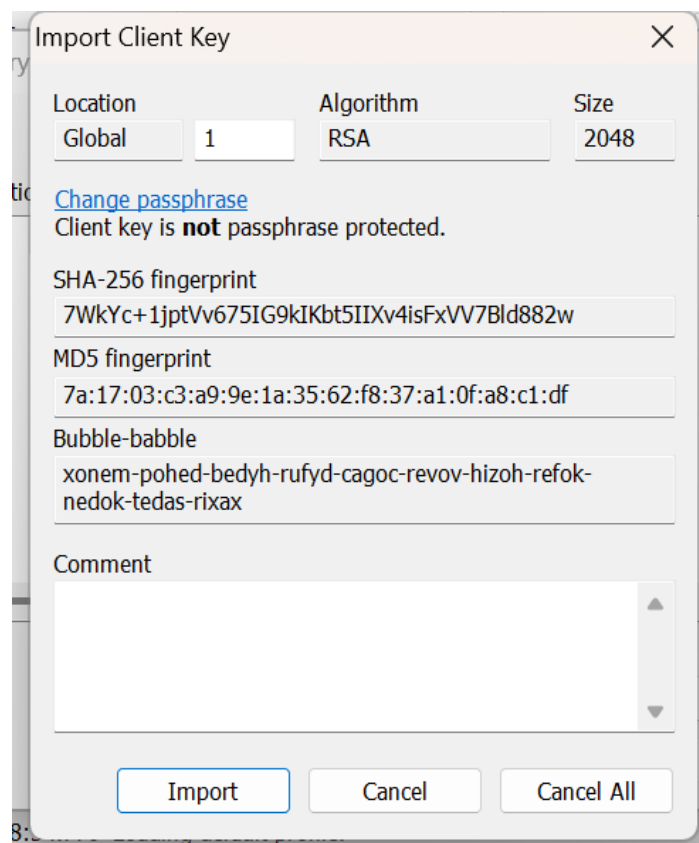
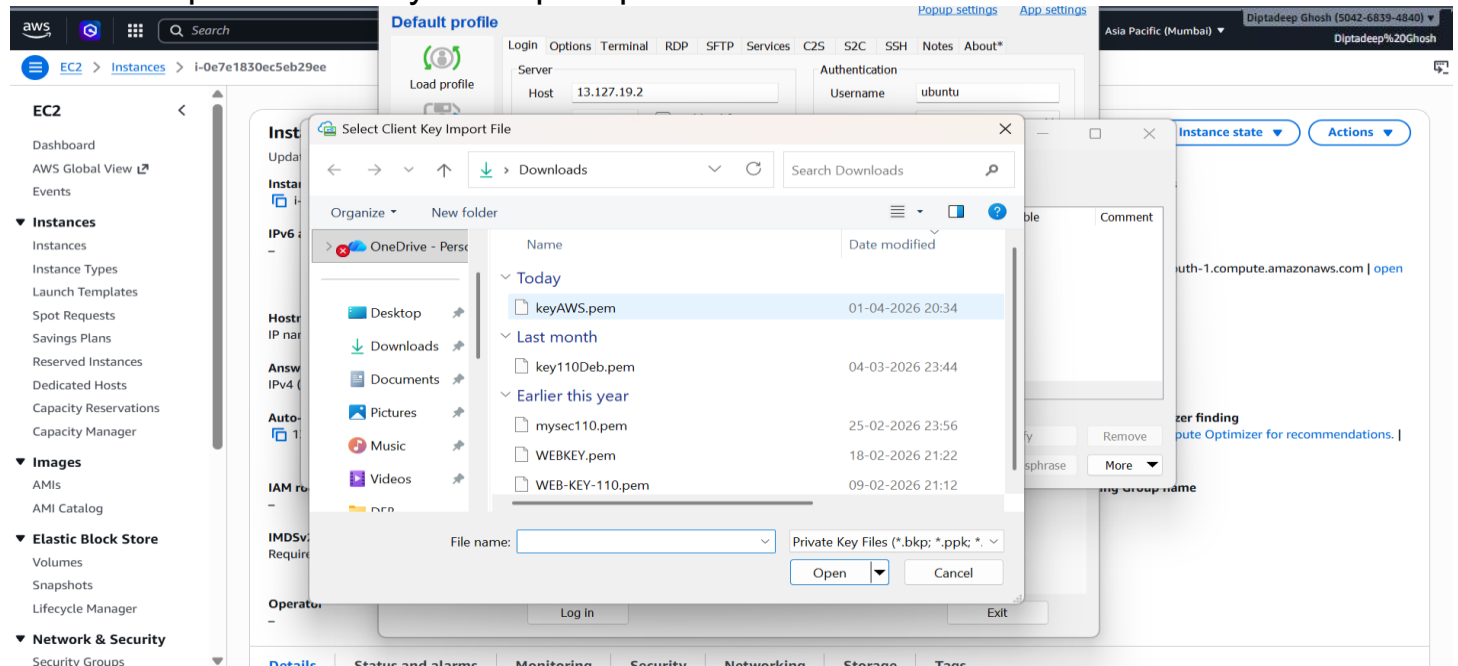
http://13.127.19.2:4000

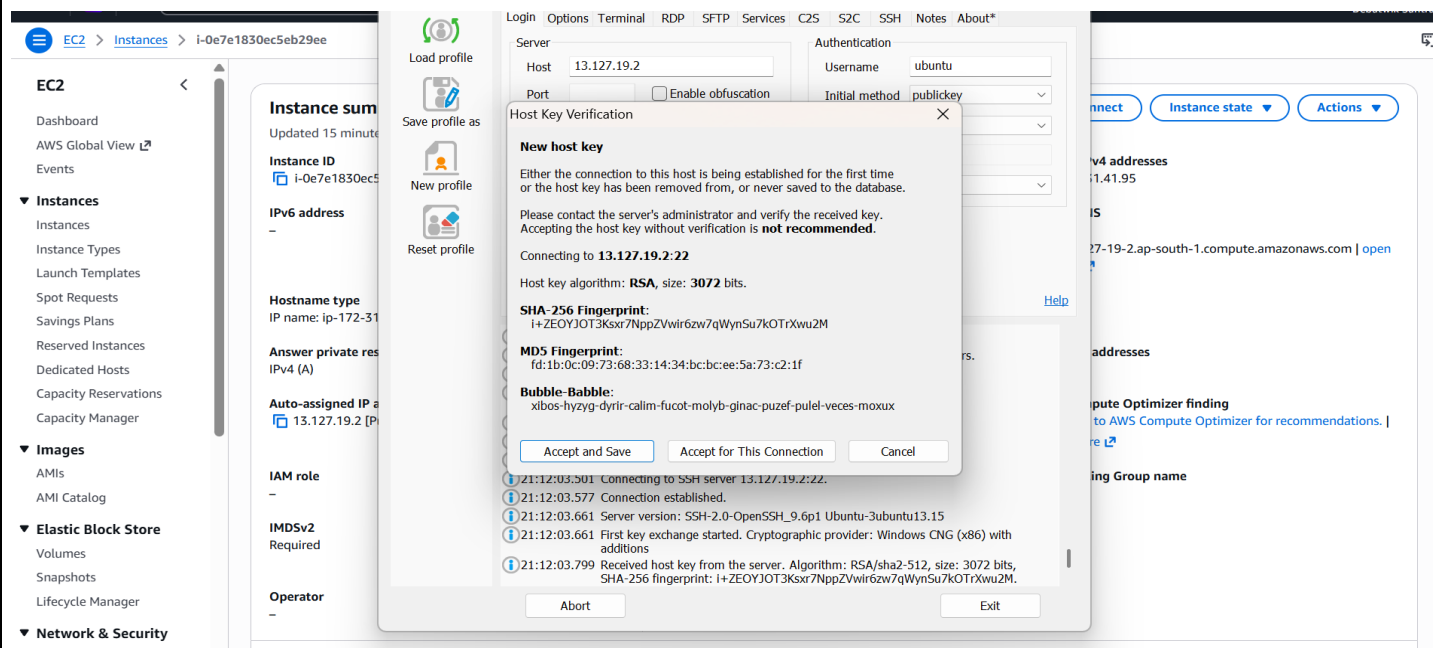
➤ It loads → Deployment Successful



STEP 9: Connect to EC2 Using Bitwise SSH

1. Open **Bitwise SSH Client**.
2. Enter:
Host: **13.127.19.2**
Username: **ubuntu**
3. Import the **keyAWS** key file.
4. Click **Login**.
5. Accept the host key when prompted.





STEP 10: Navigate to Nginx Configuration Directory

After login, open **Terminal** inside Bitvise.

First move to root directory.

```
cd /
```

Check folders.

```
ls
```

Move to **etc** directory.

```
cd etc
```

Check files.

```
ls
```

Now go to **nginx** folder.

```
cd nginx
```

List files again.

```
ls
```

Go to **sites-available** directory.

```
cd sites-available
```

Check configuration files.

```
ls
```

You will see:

```
default
```

```

ubuntu@ip-172-31-41-95:~$ cd /
ubuntu@ip-172-31-41-95:/$ ls
bin          dev          lib          lost+found  opt          run          snap        tmp
bin.usr-is-merged etc        lib.usr-is-merged media        proc         sbin        srv         usr
boot         home        lib64        mnt         root         sbin.usr-is-merged sys         var

ubuntu@ip-172-31-41-95:/$ cd etc
ubuntu@ip-172-31-41-95:/etc$ ls
ModemManager      groff          modprobe.d      screenrc
PackageKit         group          modules          security
X11                group-        modules-load.d  selinux
acpi               grub.d        mtab            sensors.d
adduser.conf       gshadow       multipath       sensors3.conf
alternatives       gshadow-     multipath.conf  services
apparmor           gss          nanorc          sgml
apparmor.d         hdparm.conf  needrestart     shadow
appport            hibagent-config.cfg netconfig       shadow-
apt                hibinit-config.cfg netplan         shells
bash.bashrc        host.conf    network        skel
bash_completion    hostname    networkd-dispatcher sos
bash_completion.d  hosts       networks       ssh
bindresvport.blacklist hosts.allow  newt           ssl
binfmt.d           hosts.deny   nftables.conf  subgid
byobu              init.d       nginx          subgid-
ca-certificates    initramfs-tools nsswitch.conf  subuid
ca-certificates.conf inputrc      opt           sudo.conf
chrony             iproute2     os-release     sudo_logsrvd.conf
cloud              iscsi        overlayroot.local.conf sudoers
console-setup      issue        pam.conf       sudoers.d
credstore          issue.net    pam.d          supercat
credstore.encrypted kernel

ubuntu@ip-172-31-41-95:/etc$ cd nginx
ubuntu@ip-172-31-41-95:/etc/nginx$ ls
conf.d          koi-utf      modules-available  proxy_params  sites-enabled  win-utf
fastcgi.conf   koi-win     modules-enabled    scgi_params   snippets
fastcgi_params mime.types   nginx.conf         sites-available uwsgi_params

ubuntu@ip-172-31-41-95:/etc/nginx$ cd sites-available
ubuntu@ip-172-31-41-95:/etc/nginx/sites-available$ ls
default
ubuntu@ip-172-31-41-95:/etc/nginx/sites-available$ sudo nano default
ubuntu@ip-172-31-41-95:/etc/nginx/sites-available$

```

STEP 11: Edit Nginx Default Configuration

Open the configuration file.

sudo nano default

Inside the file find:

location /

Modify it as follows:

location / {

proxy_pass http://localhost:4000;

proxy_http_version 1.1;

proxy_set_header Upgrade \$http_upgrade;

proxy_set_header Connection 'Upgrade';

proxy_set_header Host \$host;

proxy_cache_bypass \$http_upgrade;

}

This configuration tells **Nginx** to forward all traffic from port 80 to port 4000 where my Node.js app is running.

```

root /var/www/html;

# Add index.php to the list if you are using PHP
index index.html index.htm index.nginx-debian.html;

server_name _;

location / {
    proxy_pass http://localhost:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'Upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}

```

Save and exit:

Press

Ctrl + X

Then press

Y

Press

Enter

STEP 12: Restart Nginx Server

Now go back to main directory.

cd ..

cd ..

Restart Nginx service.

sudo systemctl restart nginx

This reloads the new configuration.

```
ubuntu@ip-172-31-41-95:/etc/nginx/sites-available$ cd ..
ubuntu@ip-172-31-41-95:/etc/nginx$ cd ..
ubuntu@ip-172-31-41-95:/etc$ sudo systemctl restart nginx
ubuntu@ip-172-31-41-95:/etc$
```

STEP 13: Access Application Without Port

Open **Browser (Incognito mode)**.

Paste only the **Public IPv4 address**:

http://13.127.19.2

Now the application will load **without :4000**.

(Previously: <http://13.127.19.2:4000>)

Now:

http://13.127.19.2



➤ **Deployment successful**

Conclusion

The project was successfully deployed on an AWS EC2 instance. Initially the application was accessed using port 4000. Nginx was configured as a reverse proxy to redirect incoming HTTP requests from port 80 to port 4000. After restarting the Nginx service, the application could be accessed using only the public IPv4 address without specifying the port number, confirming successful deployment.